A Comparative Analysis of Memory Safety, Concurrency, and Performance in Edge-AI

Timothy Clarke
University of Dundee
United Kingdom
2712139@dundee.ac.uk

1. Introduction

1.1. Background and Context

In recent years, Edge-AI (Edge Artificial Intelligence) has begun to move the deployment of AI models from centralised cloud-based servers to local devices such as sensors, mobile phones, and embedded systems. This allows for real-time processing and reduces internet bandwidth usage, making it suitable for applications where reduced latency is critical (e.g. fitness trackers, autonomous vehicles, etc.), or where connectivity is unreliable or unavailable (e.g. remote weather stations, satellite image analysis, etc.).

TODO: discuss advances in hardware acceleration

Edge-AI deployment brings challenges in terms of resource constraints, such as limited computational power, memory, and power requirements. Remote software updates to maintain Edge-AI applications also come at a cost, as they can be expensive and time-consuming, especially when dealing with a large number of devices. These challenges make language selection an important design decision for Edge-AI pipelines.

136

1.2. Problem Statement

When deploying on Edge hardware, the above listed challenges amplify the impact of programming language choice. A language's runtime model dictates memory, concurrency, and scheduling behaviour under load, which directly impacts latency, throughput, and resource usage. For example, manual memory management offers fine-grained control and increased performance, but simultaneously increases the risk of memory leaks and undefined behaviour. Conversely, memory management may be automated through garbage collection (GC) at the cost of increased latency and unpredictable latency jitter.

Selecting a language for Edge-AI pipelines is often guided by familiarity or generalised benchmarks, rather than the evaluation of runtime models under stress with the constraints of embedded hardware. There is a lack of empirical evidence of how specific memory management and concurrency models interact with backpressure policies under heavy, fluctuating loads. Additionally, resource contention and thermal throttling confounders can introduce noise that limits the validity of naive comparisons.

This dissertation addresses this gap by providing empirical evidence and an evaluation of pipelines deployed on resource-constrained hardware, with a focus on the trade-offs among Rust, C++, and Python implementations of a dual-stream Human Activity Recognition (HAR) pipeline on industry-standard Edge-AI hardware. It focuses on three confounders: (1) language runtime models, (2) backpressure policies under various loads, and (3) thermal/power throttling.

209

1.3. Research Questions and Objectives

The primary research questions are:

RQ1 **Runtime Performance:** How do the runtime models of Rust, C++, and Python influence latency, throughput, and memory consumption in a dual-stream HAR pipeline on Edge-AI hardware?

RQ2 **Backpressure Interaction:** How do the language-specific runtime models interact with different backpressure policies under varying load, and what are the trade-offs in observed deadline adherence, system stability, and allocator/GC pressure and concurrency overhead?

RQ3 **Dynamic Profiling vs. Runtime Behaviour:** To what extent do dynamic memory/concurrency profiling metrics explain the observed performance bottlenecks and trade-offs under load?

The primary research objectives are:

RO1 Implement a functionally identical dual-stream HAR pipeline in Rust, C++, and Python, ensuring optimised idiomatic implementations for each language.

RO2 Evaluate the performance of each implementation under controlled conditions, using a shared deterministic load generator, to quantify how backpressure policies and runtime models impact latency, throughput, memory consumption, and thermal/power dynamics under load.

RO3 Analyse runtime model overhead to explain performance differences, and derive empirically grounded guidance for language selection in constrained Edge-AI deployments.

166

1.4. Research Contributions

This dissertation offers the following contributions to software engineering for Edge-AI systems:

C1 **Cross-Language Runtime Evaluation:** Addressing RQ1, this work delivers an empirical comparison of how Rust, C++, and Python runtime models (memory management and concurrency) influence latency, throughput, and memory consumption on resource-constrained embedded hardware.

C2 **Interaction Analysis:** Addressing RQ2, this work provides a controlled assessment of how backpressure policies interact with runtime models under various loads, and identifies trade-offs between throughput and long-term system stability.

C3 **Root-Cause Identification:** Addressing RQ3, this work presents empirical evidence linking dynamic memory allocation churn, GC pause duration, and scheduling overhead to system latency and throughput.

C4 **Evidence-Based Guidelines:** Addressing findings across all research questions, this dissertation offers empirically grounded recommendations for language selection in real-time, multi-stream edge deployments.

122

1.5. Scope and Limitations

The focus of this dissertation is on the interaction of three language runtime models (Rust, C++, and Python) with system latency and throughput, using standardised, idiomatic implementations of a dual-stream HAR pipeline on industry-standard Edge-AI hardware. The scope is limited to a standardised implementation representative of real-world multi-modal processing, with confounders limited to thermal/power throttling and queue-based backpressure policies.

The pipeline architecture, deterministic load generator, and backpressure mechanisms are designed for cross-platform compatibility.

However, AI acceleration, thermal behaviour, and power management are fundamentally SoC-dependent. Consequently, the profiling and acceleration tools used during evaluation (e.g., NVIDIA tegrastats, TensorRT) are specific to the Jetson Orin Nano platform and cannot be directly applied across other edge devices.

115

1.6. Dissertation Outline

The remainder of this dissertation is structured as follows: **Chapter 2** reviews related work, the runtime models of the target languages, and backpressure policies. **Chapter 3** details the methodology, including the hardware and software setup, backpressure interfaces, and profiling toolchains to collect metrics. **Chapter 4** describes the implementation of the HAR pipeline in each language, highlighting language-specific optimisations and challenges. **Chapter 5** presents the results, including performance metrics, memory allocation and GC pressure, and backpressure outcomes under varying loads. **Chapter 6** discusses the findings, and limitations of the study. Finally, **Chapter 7** concludes by addressing the research questions, providing practical recommendations for language selection in Edge-AI contexts, and suggesting directions for future work.

113

884

2. Literature Review

TODO:

- Synthetic HAR data generation.
- Backpressure policies.
- Runtime models of Rust, C++, and Python.
- Performance of Rust, C++, and Python in Edge-AI contexts.
- Thermal and power management on embedded hardware.
- Profiling tools for Edge-AI hardware.
- Previous comparative analyses of programming languages for Edge-AI.
- Previous work on language runtime models and backpressure policies.
- Best practices for Rust real-time accuracy.

60

3. Methodology

3.1. Hardware and Software Environment

3.1.1. Hardware Stack

The NVIDIA Jetson Orin Nano Super was utilised as the target Edge-AI platform. It is a high-performance system-on-module (SoM) designed for Edge-AI development and allows complex AI workloads and multi-stream pipelines to run efficiently. The developer kit includes a carrier board with I/O interfaces (e.g., USB, Ethernet, DisplayPort), a MicroSD card slot for booting, and the Jetson Orin Nano module itself which includes the following features [1]:

- 6-core Arm Cortex-A78AE 64-bit CPU for general-purpose concurrent processing
- up to 67 TOPS (Tera Operations Per Second) of AI performance
- 8 GB of 128-bit LPDDR5 memory with a bandwidth of 102 GB/s
- 1024 CUDA cores for general-purpose GPU computing
- 32 Tensor cores for AI acceleration

A Waveshare IMX219-160 Camera Module [2] was used to deliver the RGB video stream, configured to capture at 1920×1080 RGB frames at 30 FPS, and connected via MIPI CSI-2 (Mobile Industry Processor Interface Camera Serial Interface 2). The images captured by the camera's 160° FOV required undistortion

The camera captures images with a field of view (FOV) of 160 degrees, making it suitable for capturing a wide area for human activity recognition.

A Bosch Sensortec BMI088 IMU Shuttle Board 3.0 [3] was used to provide inertial measurement data, configured to capture 6-axis data, and connected via SPI for maximum throughput. The BMI088 combines a 3-axis accelerometer and a 3-axis gyroscope, providing two complementary data streams at up to 1.6 kHz (accelerometer) and 2.0 kHz (gyroscope).

To ensure that disk I/O did not cause bottlenecks or confound performance comparisons, all implementations were executed from a 1TB Samsung 990 PRO PCIe 4.0 NVMe M.2 SSD [4]. A SanDisk "High-Endurance" microSD Card (64GB, Class 10/U3) [5] was only used for initial device installation and bootloading, and was unmounted after boot to prevent any background I/O (such as writing logs) from interfering with performance measurements.

306

3.1.2. Software Stack

The Jetson was flashed with NVIDIA's JetPack 7.1 SDK [6], which includes Jetson Linux 38.4 [7] (which uses the Ubuntu 24.04-based root file system), and CUDA 13.0.0, cuDNN 9.12.0, and TensorRT 10.13.3.9 for AI inference and acceleration. The software stack versions were selected as the most recent stable releases at the time of development, and were used for all implementations to ensure a consistent baseline for comparison. TODO: Add ONNX version

CUDA (Compute Unified Device Architecture) [8] provides a parallel execution environment and programming model for heterogeneous computing systems with NVIDIA GPUs, using Single Instruction Multiple Threads (SIMT) architecture. In CUDA, the CPU is referred to as the *host*, and the GPU is referred to as the *device*. CUDA clients are responsible for managing the transfer of data between *host memory* and *device memory*.

CUDA allows developers to write a *kernel* function that is launched asynchronously from the host code and executed in parallel across many threads (using different data) on the device, enabling high-performance computing for AI workloads. The kernel *threads* are grouped into *thread blocks*, which in turn are grouped into a *grid*. Each thread block is split into *warps* of 32 threads that are executed in lock step on a single device *Streaming Multiprocessor* (SM).

cuDNN (CUDA Deep Neural Network) [9] is a GPU-accelerated library of primitives for deep neural networks, that sits on top of CUDA and runs on the device to provide higher-level abstractions and optimised implementations of common deep learning operations (e.g., normalisation, matrix multiplication, softmax, etc.).

TensorRT [10] is responsible for compiling an **ONNX** (Open Neural Network Exchange) [11] model into a *.engine* file, optimised to run on the Jetson GPU.

All implementations interact with the **ONNX Runtime** to load and execute the AI model, which is responsible for the data transfer and inference orchestration on the host, and hands off responsibility to TensorRT for inference execution on the device. To ensure a consistent baseline, the same optimised *.engine* file was used across all three implementations.

Performance and overhead of the language runtime models was measured at the boundary of the host/device memory separation, where the CPU manages the runtime model and the SIMT architecture runs the AI workloads on the GPU. This prevents confounding the results with hardware latency, and provides a clearer comparison of how each language's runtime model performs under load and backpressure.

A Docker container, based on NVIDIA's official *l4t-base* image (version), was used to prevent host updates to ensure that the software toolchains and environment variables remained consistent for all implementations. TODO: Add details about container configuration, e.g., volumes, GPU and device access, privileged mode (if used), etc. While Docker introduces some performance overhead, it was

considered acceptable to ensure a consistent and reproducible environment for all implementations.

The latest stable releases of the language toolchains were used for all implementations: Rust version, C++20 with GCC version, and Python version.

480

3.1.3. Deterministic Load Generator

To reliably compare the performance of the three implementations, a synthetic load generator was developed to create reproducible and deterministic simulated sensor data. This ensures that the behaviour of each implementation can be compared using the same baseline data, and that differences in performance can be attributed to the runtime models and backpressure policies, and not input variability.

The load generator produces three streams of data to shared memory buffers for consumption by the HAR pipelines: TODO: check how sensors provide data — may need harness to connect with API and also copy to shared memory (1) an RGB video stream to simulate the camera, (2) a 6-axis inertial measurement stream to simulate the accelerometer, and (3) a second 6-axis inertial measurement stream to simulate the gyroscope.

Using shared memory allows for low-latency communication, and will not block the generator if the pipelines are backpressured. The buffers were allocated a fixed capacity to allow for backpressure, and the pipelines were considered saturated when the buffers were full which would trigger the configured backpressure policy.

The generated image data was random noise. Each image was created as an array of RGB pixel values with dimensions of 1920x1080 to match the sensor data, and each pixel's red, green, and blue values were assigned random whole numbers in the range $[0, 255]$.

To generate the IMU data, mathematical functions were used to create data similar to that produced by real-world movements (e.g., sine waves to simulate smooth motion, etc.), while still being deterministic and reproducible.

TODO: Check previous work for synthetic HAR data generation.

To allow backpressure policies to be evaluated under varying load, the generator accepts a *load* parameter that dictates the speed at which data is produced by acting as a divisor of the baseline sensor intervals. For example, a value of 1.0 produces data at the same rate as the sensors: 30 FPS for the camera (1 frame every 33.3 ms), and 1.6 kHz and 2.0 kHz for the accelerometer and gyroscope respectively (0.625 ms and 0.5 ms intervals respectively). A value of 2.0 produces data twice as fast, 0.5 produces data at half the speed, and so on. 100% saturation of the pipelines was determined by adjusting the load argument until the pipelines were consistently backpressured (see Section 3.1.4).

A hard-coded seed for each sensor ($\text{rgb} = 42, \text{accel} = 43, \text{gyro} = 44$) was used to create three deterministic data-streams to ensure the same generated data was fed into each implementation. Because the AI inference is only a repeatable workload to determine the performance of the runtime models, and we are not concerned about prediction accuracy, the generated data was not designed to be realistic. This kept the load generator implementation simple, and reduced latency and overhead that could confound the results.

The load generator was written in Rust, which is a performance-oriented language, and is memory-safe without a GC that may introduce latency spikes. For maximum timing accuracy, `nix::time::clock_nanosleep` was used to implement the timing of the data generation, the generator was pinned to one CPU core to prevent jitter, and the process was given a real-time scheduling policy.

Before using the load generator, the HAR pipelines were tested with real sensor data to ensure that they were functionally correct and optimised.

536

3.1.4. Backpressure Policies

Bounded backpressure policies are implemented in each language-specific runtime model. In a typical backpressure implementation, when a *consumer* is saturated (i.e., the buffer is full), the active backpressure policy is triggered to slow the flow of data from the *producer* to prevent unbounded memory demand and system instability.

The backpressure policies were implemented in the pipelines using two shared memory buffers per data stream: (1) an unbounded *producer buffer* in shared memory for the load generator to write data into, allowing it to produce data at a consistent rate, and (2) a *consumer buffer* implemented idiomatically for the pipelines to read data from for processing, with a fixed capacity to trigger the backpressure policy when full. Backpressure is implemented only in the pipeline on the consumer buffer, forcing each language runtime model to handle concurrency, memory allocation, and scheduling within realistic constraints and allowing us to evaluate RQ2. A *bridge* in the pipeline is responsible for copying data from the producer buffer to the consumer buffer, and for triggering the backpressure policy when the consumer buffer is full.

Each pipeline uses language-specific idiomatic implementations of the backpressure policies: Rust uses TODO, C++ TODO, and Python TODO.

Four backpressure policies where implemented: (1) *bounded queue*, which blocks the producer when the buffer is full until space is available, (2) *drop-oldest*, which drops the oldest data in the buffer to make room for new data when the buffer is full, (3) *drop-newest*, which drops the newest data when the buffer is full, and (4) *exponential-back-off*, which waits a short time before retrying to produce data when the buffer is full, with the wait time doubling with each retry. TODO: Review literature for common backpressure policies and confirm these are the most relevant for Edge-AI pipelines.

TODO: Add detail of how saturation was determined.

303

3.1.5. Profiling and Metrics

3

3.1.6. Statistical Analysis

2

1641

4. Implementation

4.0.1. Model Fusion

TODO:

- Two AI models.
- Late fusion.
- Zero On Hold (ZOH) for IMU data.
- Sensor data interpolation not used to simulate data between sensor updates, removing number precision as a confounder.

32

5. Results

6. Discussion

7. Conclusion

Total words: 2647

References

[1] "NVIDIA Jetson Orin Nano Super Developer Kit." Accessed: May 09, 2026. [Online]. Available: https://nvdam.widen.net/s/zkfqjmtds2/jetson-orin-datasheet-nano-developer-kit-3575392-r2

[2] "IMX219-160 Camera." Accessed: May 09, 2026. [Online]. Available: https://www.waveshare.com/wiki/IMX219-160_Camera

[3] *BMI088, 6-axis Motion Tracking for High-performance Applications.* Accessed: May 09, 2026. [Online]. Available: https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmi088-ds001.pdf

[4] "Samsung 990 Pro NVMe SSD." Accessed: May 09, 2026. [Online]. Available: https://www.samsung.com/uk/memory-storage/nvme-ssd/990-pro-1tb-nvme-pcie-gen-4-mz-v9p1t0bw/

[5] "SanDisk High Endurance microSD Card." Accessed: May 09, 2026. [Online]. Available: https://documents.sandisk.com/content/dam/asset-library/en_us/assets/public/sandisk/product/memory-cards/high-endurance-uhs-i-microsd/data-sheet-high-endurance-uhs-i-microsd.pdf

[6] "NVIDIA JetPack SDK Downloads and Notes, JetPack 7.1 with Jetson Linux 38.4." Accessed: May 09, 2026. [Online]. Available: https://developer.nvidia.com/embedded/jetpack/downloads

[7] *NVIDIA Jetson Linux Release Notes, Version 38.4.0 GA.* Accessed: May 09, 2026. [Online]. Available: https://docs.nvidia.com/jetson/archives/r38.4/ReleaseNotes/Jetson_Linux_Release_Notes_r38.4.pdf

[8] *CUDA Programming Guide, Version 13.2.* pp. 4–11. Accessed: May 09, 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/pdf/cuda-programming-guide.pdf

[9] *NVIDIA cuDNN Developer Guide, Version 8.9.0.* Accessed: May 10, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/cudnn/archives/cudnn-890/pdf/cuDNN-Developer-Guide.pdf

[10] "NVIDIA TensorRT Documentation, Version 10.13.3." Accessed: May 10, 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/10.13.3/index.html

[11] "ONNX 1.22.0 Documentation." Accessed: May 10, 2026. [Online]. Available: https://onnx.ai/onnx/index.html